Experiment No: 12

Aim: Case Study: Distributed File System (AFS/CODA)

Theory:

Case Study: Andrew File System (AFS)

Introduction

The Andrew FileSystem(AFS) is a distributedfilesystem developed at CarnegieMellon University(CMU) as part of the Andrew Project in the 1980s. AFS was designed to scale efficiently, support file sharing across a wide-area network, and ensure dataconsistency and security.

It is one of the earliest examples of a successful DFS and inspired future systems like Coda, HDFS , and even modern cloud storage platforms.

AFS Architecture and Components

AFS follows a client-server architecture with some unique characteristics.

1. Clients and Servers

- Servers store file volumes.
- Clients access these files through a local cache, giving the illusion of local file access.

2. Volumes

- A volume is a logical collection of files and directories.
- Each volume can be moved or replicated across servers without affecting users.

3. Cache Manager (on Client)

- Locally caches files upon access.
- If the file is unchanged on the server, the cache is reused.
- Reduces server load and improves performance.

4. File Identifiers

- AFS uses globally unique file identifiers.
- Paths like /afs/your-university.edu/ allow seamless access across institutions.

5. Authentication via Kerberos

- AFS uses the Kerberos authentication protocol to verify user identity and provide secure access.

6. Callback Mechanism

- When a client reads a file, it gets a callbackpromise.
- If another client modifies the file, the server inv alida testhe first client's cache.

How AFS Works – Step-by-Step

1. Userlogs in and authenticates using Kerberos.
2. The client requests a filefrom the AFS server.
3. Server sends file and a callbacktoken .
4. File is cached locallyon the client.
5. Any further access is localuntil the server signals a change.
6. When the file is modified remotely, the server revokes the callback, prompting the client to fetch a new copy.

Real-World Use Case: University File Sharing

Problem:

Universities needed to manage and share educational resources, research documents, and student files efficiently across departments and campuses.
AFS Solution:

- Provided a global namespace (/afs/university.edu/department/)
- Allowed seamless access from any authorized machine.
- Reduced load on servers with efficient caching.
- Enabled collaboration among researchers and faculty across locations.

Outcome:

- Significant improvement in performance.
- High availability and better user experience.
- Easier administration and data consistency.

Key Features of AFS

Feature	Description
Scalability	Supports thousands of users and large volumes.
Location Transparency	Users don't need to know where files are physically stored.
Efficient Caching	Reduces network traffic and improves speed.
Callback Mechanism	Ensures cache consistency.

Feature	Description
Volume Management	Easy relocation and replicationoffile groups.
Secure Authentication	Via Kerberos.
Global Namespace	Unified file system across institutions.

Advantages of AFS

- High scalability
- Strong caching system for performance
- Secure user authentication
- Transparent file access and management
- Efficient server load handling

Limitations of AFS

Limitation	Description
Complex Setup	Requires deep understandingand admin effort.
Write Latency	Due to delayed synchronization of writes.
Single Point of Failure	Without replication, server failures can disrupt access.
Less suited for frequent writes	Cache may delay write propagation.
Not optimal for mobile/disconnected clients	Unlike Coda, AFS doesn't support disconnected operation well.

Legacy and Influence

AFS heavily influenced the design of:

- Coda File System – added disconnected operation and conflict resolution.
- HDFS (Hadoop) – adopted similar ideas for block storage.
- Modern cloud storage services like Google Drive, OneDrive, and Dropbox.

AFSisstill inusetodayinenterpriseand academic institutions(inits open-sourceform: OpenAFS).

Conclusion

The Andrew File System was a milestone in distributed computing, offering a highly scalable, secure, and consistent file-sharing solution for large organizations.

Its core ideas are foundational to modern distributed and cloud file systems.